

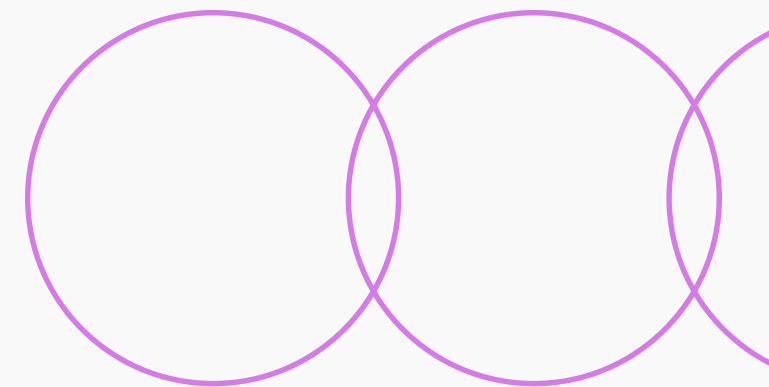
Evaluacion SDLC



Conceptos Básicos del SDLC

Fases del Desarrollo de Software

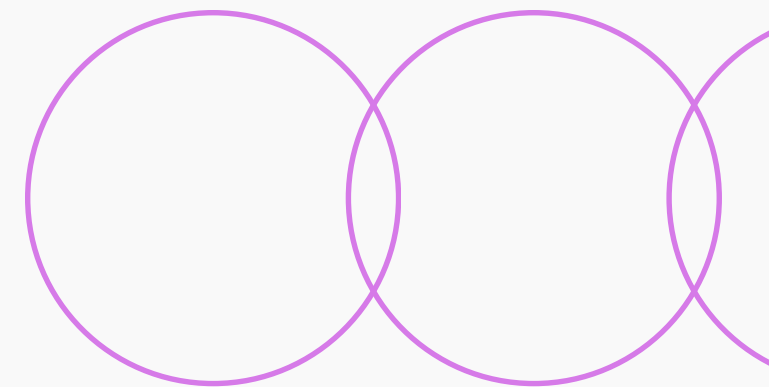
El **SDLC** consiste en un marco estructurado que abarca siete fases: planificación, análisis de requisitos, diseño, implementación, pruebas, despliegue y mantenimiento, cruciales para el desarrollo eficiente de software.



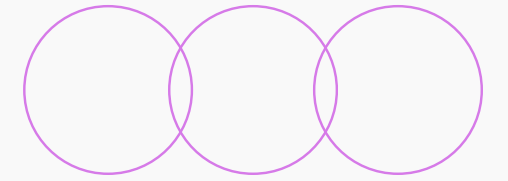
Importancia del Alcance

Definición Clara del Alcance

Definir el **alcance del proyecto** es crucial para el éxito. Establece límites claros, evita cambios no controlados y ayuda a mantener el enfoque del equipo dentro de los tiempos y presupuesto.



Clasificación de Requisitos



Funcionales y No Funcionales

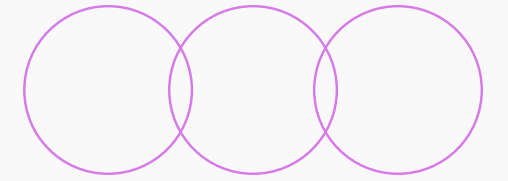
Requisitos Funcionales

Los **requisitos funcionales** especifican las funciones y características que el software debe tener. Estos requisitos definen el comportamiento del sistema y son esenciales para cumplir con las expectativas del usuario. Ejemplos incluyen la capacidad de iniciar sesión, realizar pagos y generar reportes. Aseguran que el software opere de manera efectiva y satisfaga las necesidades del cliente.

Requisitos No Funcionales

Los **requisitos no funcionales** se centran en aspectos como el rendimiento, la escalabilidad y la seguridad del sistema. Estos requisitos son cruciales porque afectan la experiencia del usuario y la calidad del software. Por ejemplo, especificaciones sobre tiempo de respuesta, capacidad de carga y protección de datos son todos requisitos no funcionales que garantizan que el software funcione de manera óptima en diferentes condiciones.

Diseño del Sistema



HLD y LLD Explicados

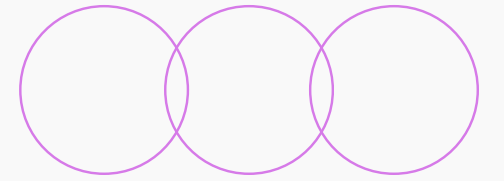
Alto Nivel (HLD)

El Diseño de Alto Nivel (HLD) proporciona una visión general de la arquitectura del sistema. Este enfoque resalta los módulos principales, las tecnologías utilizadas y cómo se comunican entre sí, facilitando una comprensión global del sistema y sus componentes clave.

Bajo Nivel (LLD)

El Diseño de Bajo Nivel (LLD) detalla cada módulo en profundidad, incluyendo clases, métodos y estructuras de datos específicas. Este diseño actúa como una guía técnica que orienta a los desarrolladores en la implementación del código de manera precisa y efectiva.

Principios de Diseño



Código Limpio y Estructurado

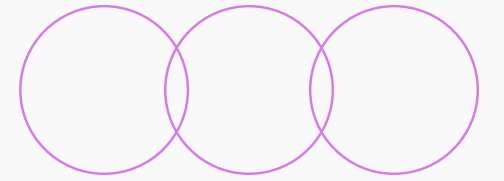
Patrón MVC

El **patrón MVC** (Modelo-Vista-Controlador) es una arquitectura que separa la lógica de negocio, la interfaz de usuario y la gestión de entradas. Esta separación permite un desarrollo más organizado, facilitando la implementación de cambios y el mantenimiento del sistema. Al dividir las responsabilidades, cada componente puede evolucionar sin afectar al resto, lo que mejora la escalabilidad del software.

Código Limpio

Código limpio se refiere a la práctica de escribir código que sea fácil de leer y mantener. Esto incluye el uso de nombres descriptivos para variables y funciones, funciones cortas que cumplan una sola responsabilidad y la eliminación de duplicados. Un código bien estructurado no solo facilita la comprensión, sino que también reduce el riesgo de errores, haciendo que cada desarrollo futuro sea más eficiente.

Pruebas y Calidad



Importancia de las Pruebas

Pruebas Unitarias

Las **pruebas unitarias** son esenciales para garantizar que cada función o método de software funcione correctamente. Estas pruebas permiten validar la lógica de cada componente de forma aislada, **asegurando** que no existan errores en el código antes de ser integrado con otros módulos del sistema.

Beneficios de las Pruebas

Implementar un enfoque de pruebas unitarias proporciona numerosos beneficios, como la detección temprana de errores, la mejora de la calidad del código y la **facilitación** de cambios futuros. Al contar con una suite de pruebas bien definida, los desarrolladores pueden realizar modificaciones con confianza, sabiendo que cualquier regresión será identificada rápidamente.

Despliegue y Mantenimiento

CICD

La **Integración Continua** y **Entrega Continua** automatizan el proceso de integración, pruebas y despliegue para lanzar cambios de manera rápida y segura.

Mantenimiento Correctivo

El **mantenimiento correctivo** se enfoca en corregir errores o fallas existentes en producción, asegurando el funcionamiento adecuado del sistema.

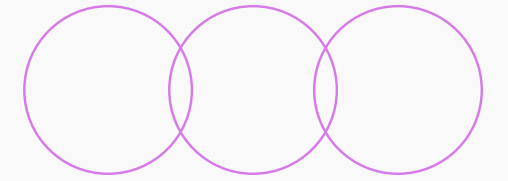
Docker

Docker permite empaquetar aplicaciones con todas sus dependencias, asegurando un funcionamiento consistente en cualquier entorno, facilitando el despliegue.

Mantenimiento Evolutivo

El **mantenimiento evolutivo** consiste en agregar nuevas funcionalidades o mejoras a las existentes, adaptándose a las necesidades cambiantes de los usuarios.

Análisis de Caso



Fallo de Escalabilidad

Tipo de Requisito

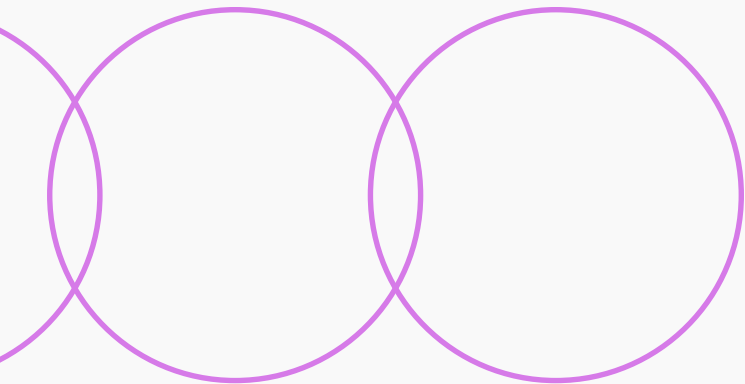
El fallo de escalabilidad se clasifica como un **requisito no funcional**. Esto se refiere a las expectativas sobre el rendimiento del sistema, como su capacidad para manejar múltiples usuarios simultáneamente. En este caso, no se definió ni garantizó la capacidad del sistema para soportar la carga alta de usuarios, lo que resultó en fallos durante su operación.

Fase de Detección

La fase donde debió detectarse este problema es durante el **análisis de requisitos**. Es fundamental identificar la capacidad máxima de usuarios que el sistema puede acomodar. Además, es crucial validar este aspecto en la fase de pruebas mediante **pruebas de carga** y estrés antes del despliegue para asegurar que el sistema funcione correctamente bajo condiciones reales.

¿Cuáles son los próximos pasos?

Reflexionemos sobre nuestro **proceso de SDLC** y cómo avanzar



SOFTWARE DEVELOPMENT PROCESS 1FC

COINTRRATOR PETILLE FIOPS

